

- I'll use the PDF workflow again, focusing on the handout's Project 2 description, its surrounding design principles, evaluation guidance, and any details that should correct or sharpen the earlier plan.

- **Explored**

- └ Read SKILL.md (pdf:pdf skill)

- **Ran** pdftinfo

- └ /Users/Checkout/Downloads/FDE_Portfolio_Handout_Aishwarya_Srinivasan.pdf && mkdir -p tmp/pdfs/fde-handout && pdftotext -layout /Users/Checkout/Downloads/FDE_Portfolio_Handout_Aishwarya_Srinivasan.pdf tmp/pdfs/fde-handout/handout.txt && wc -l -w -c tmp/pdfs/fde-handout/handout.txt && sed -n '1,260p' tmp/pdfs/fde-handout/handout.txt

- └ YOUTUBE COMPANION · PORTFOLIO GUIDE

- └ ... +256 lines (ctrl + t to view transcript)
exceptions nobody mentioned during the sales process. Imagine a risk platform onboarding a new enterprise customer who needs to send CRM, billing, and support data into your product. Their schemas

- **Explored**

- └ Read handout.txt

- **Ran** pdftoppm -png -r 120

- └ /Users/Checkout/Downloads/FDE_Portfolio_Handout_Aishwarya_Srinivasan.pdf tmp/pdfs/fde-handout/page >/dev/null 2>&1 && ffmpeg -hide_banner -loglevel error -start_number 1 -i tmp/pdfs/fde-handout/page-%d.png -vf "scale=240:-1,tile=3x3:padding=8:margin=8" -frames:v 1 tmp/pdfs/fde-handout/contact.png && file tmp/pdfs/fde-handout/contact.png

- └ tmp/pdfs/fde-handout/contact.png: PNG image data, 752 x 965, 8-bit/color RGB, non-interlaced

- **Viewed Image**

- └ tmp/pdfs/fde-handout/contact.png

- **Viewed Image**

- └ tmp/pdfs/fde-handout/page-4.png

- **Viewed Image**

- └ tmp/pdfs/fde-handout/page-8.png

- I've read the complete nine-page handout, concentrating on Project 2 and the surrounding FDE guidance.
/Users/Checkout/Downloads/FDE_Portfolio_Handout_Aishwarya_Srinivasan.pdf

The most important clarification is this:

> We are not merely building a GitHub issue classifier. We are building an operational intake-to-resolution workflow, and GitHub issues are the concrete domain used to demonstrate it.

What the project actually does

A messy request arrives through a form, email, support system, or ticketing API. The system then:

1. Reads and structures the request.
2. Looks up any necessary context.
3. Classifies its category, urgency, owner, and likely next action.
4. Runs deterministic company policies outside the model.
5. Chooses an action:
 - Ask for missing information
 - Draft a response
 - Route to the correct team
 - Escalate to a human
6. Pauses for human review where appropriate.
7. Executes or records the approved action.
8. Preserves the state and complete decision history.

The output is therefore a business decision and an action—not a paragraph from a chatbot.

How GitHub issues fit

Our proposed GitHub version is a specialization of that workflow:

- A GitHub issue is the incoming request.
- The title and description are the unstructured input.
- Existing labels, ownership information, repository rules, and perhaps related issues provide context.
- The model proposes category, urgency, owner, and next action.
- Python policy rules enforce non-negotiable behavior.
- A reviewer can correct and approve the proposal.

- The system could then draft a comment, apply labels, assign a team, or flag an issue for escalation.
- Historical closed issues provide evaluation data because maintainers already made many of these decisions.

This makes elastic/kibana a possible dataset and demonstration domain, but the project itself is not “a Kibana classifier.”

What the handout requires that the earlier plan understated

The previous conversation concentrated heavily on classification and evaluation. The handout expects a little more:

- An actual intake path, such as a web form, GitHub API import, or webhook simulation.
- A context lookup step where useful.
- A genuine downstream action, even if public demo mode only records what it would have done.
- Persistent human approval state.
- Failure handling for invalid model output, failed integrations, missing context, and out-of-policy requests.
- A complete vertical slice from incoming request to approved operational outcome.

Historical Kibana issues should serve primarily as the evaluation corpus. We should also provide a small live workflow where someone can submit or select an issue and observe it move through the system.

Model versus deterministic code

This is the conceptual heart of the project.

The model handles ambiguous judgment:

- Understanding messy language
- Inferring category
- Suggesting urgency
- Identifying the likely component and owner
- Drafting a response
- Explaining its recommendation

Ordinary code handles guarantees:

- Mandatory security escalation
- Severity floors

- Required-field checks
- Confidence thresholds
- Allowed workflow transitions
- Approval requirements
- Audit logging
- Whether an action may actually execute

The model proposes; policy code and people retain control.

The handout includes policy_basis in the model schema while also saying policy must remain outside the model. The safest interpretation is:

- The model may explain what signals it noticed.
- Deterministic code records the authoritative rule_id, policy decision, and override reason.

Human review has two meanings

These should not be confused:

- **Approval checkpoint:** A reviewer inspects or edits a proposed action before the workflow continues.
- **Escalation action:** The request is routed to a specialist because it is critical, sensitive, or cannot be handled automatically.

A request could be routed to a normal team after approval, or escalated to a security specialist. “Human review” should therefore be a workflow state, while escalate is a business action.

How success is measured

The handout explicitly names:

- Routing accuracy
- False escalations
- Missed escalations
- Tool/integration success
- Human-intervention rate

Missed escalations are the dangerous failure, but this is not the only metric. We must also demonstrate that the complete workflow works: intake, context lookup, structured output, policy checks, state transitions, approval, and action execution.

The earlier evaluation recommendations—temporal holdout, simple baselines, model/rules ablations, raw counts, confidence intervals, label-noise review, and real failure examples—remain strong additions.

What is essential versus optional

Essential:

- Explicit workflow
- Typed model output
- Deterministic policy layer
- At least one real or simulated downstream action
- Persistent human approval
- Audit history
- Historical replay and failure analysis
- Deployed vertical slice

Optional:

- LangGraph
- Multiple intake channels
- Complex multi-agent behavior
- A task queue
- Elaborate frontend
- Automatically modifying real public GitHub issues

The clearest one-sentence definition is:

> An AI-assisted GitHub issue-triage system that turns messy issue reports into structured, policy-checked actions, keeps humans in control, and proves its reliability by replaying real
> historical issues.

That is now my working understanding of what we should build.

Worked for 1m 23s · done 12:25 PM

– Conversation recap

The nine-page FDE portfolio handout was fully reviewed, focusing on Project 2 and the surrounding FDE guidance. The requested review is complete.

› Okay. Now, based on your understanding of what you read, the chat conversation of the previous model, and the handout, I'm guessing that you got a concrete idea of what we are trying to

build. The first thing is, I need you to flag the things that you thought are wrong here, need to change, or need a second thought in the plan.

- The plan has a strong core, but I would not start implementing it unchanged. Several assumptions were promoted to “locked decisions” too early.

Must change before coding

Current plan	Problem	Recommended
correction		
Build a classifier over historical Kibana issues component, not the complete	That is only the evaluation Build two connected paths: a live intake-and-action project described in the handout	workflow,
plus an offline historical replay harness		
Four routes include “held for human review” checkpoint, not one of the four	Human review is a workflow Keep the four actions as ask_info, draft_reply, route, and business actions	escalate; separately
decide whether an action requires	approval	
“Nothing happens until a human approves” alongside	These statements conflict	
Define a risk matrix: low-risk actions may execute automatic routing	automatically;	
uncertain or high-impact actions require	approval	

Model outputs policy_basis This conflicts with “policy stays outside the model” Let the model return observed evidence or a short explanation; deterministic code records the authoritative rule_id, decision, and override reason

Model supplies a confidence number LLM self-reported confidence is generally not reliable Derive review thresholds from validation performance, enough to control automation consistency, schema validity, and class-specific error rates—not the model’s claimed confidence alone

Closed-issue labels are treated as perfect ground truth Repository labels are incomplete, inconsistent, modified Treat labels as noisy reference decisions and audit a sample manually over time, and sometimes applied using information that was unavailable when the issue arrived

Temporal split by closure date The system makes a decision when an issue arrives, not when Split chronologically by created_at and ensure the model it closes sees only information available at intake time

“All critical issues” placed into a 1,000-item sample This creates an unrealistic class distribution and makes Maintain two evaluations: a naturally distributed overall metrics misleading operational test set and a separate critical/safety challenge set

impact:critical maps directly to security/P1 Product impact, urgency, and security sensitivity are Verify Kibana's actual label definitions and evaluate related but not identical urgency and security escalation separately

Security keywords guarantee safety Keyword rules can be evaded and can create false positives, Use rules to force review or impose a severity floor, not especially because "Elastic Security" is a product name to claim that the system has correctly identified every vulnerability

The largest conceptual gap

The previous plan blurred three different things:

1. The model's proposed business action

- Ask for information
- Draft a response
- Route
- Escalate

2. The workflow's processing state

- Received
- Classified
- Policy checked
- Awaiting review
- Approved
- Action completed
- Failed

3. The approval policy

- Auto-execute
- Require reviewer approval
- Prohibit automatic execution

These need separate fields. Otherwise "escalated," "held for review," and "approved" will become mixed together in the database and evaluation.

The dataset needs considerably more care

The choice of Kibana is promising, but it is not yet justified strongly enough to be final.

Questions that need verification:

- Does impact:critical really represent the kind of P1 outcome we intend?
- At what point in an issue's lifecycle is that label applied?
- Can multiple impact labels appear on one issue?
- Are bug, enhancement, docs, and question labels consistently applied?
- Are Team:* labels owners, affected components, or both?
- How many issues have all the labels required to evaluate category, urgency, and routing together?
- How has the taxonomy changed since 2023?

There is also a serious leakage risk. If we use comments, final labels, assignments, closure reason, or other later metadata as model input, the system may see clues created during the

very triage process it is supposed to predict. Evaluation input should usually be limited to what existed at intake: title, initial body, author metadata where appropriate, and context that would genuinely have been available then.

The proposed 50-issue label audit is useful, but the earlier statement that disagreement establishes a "label-noise ceiling" is mathematically too strong. One person disagreeing with

maintainers does not reveal the true maximum accuracy. A better approach is:

- Independently label a sample using a written rubric.
- Record justification.
- Adjudicate disagreements.
- Report agreement and disagreement patterns.
- Call the result a label-quality audit, not an absolute performance ceiling.

Evaluation needs restructuring

A single holdout set is insufficient because prompt, rule, and threshold development will gradually leak information from it.

Use three chronological partitions:

- Development set: inspect freely and use to design the system.
- Validation set: choose prompts, mappings, rules, and review thresholds.
- Final test set: keep sealed until the system is stable.

The evaluation should score each layer separately:

- Structured-output/schema success
- Category performance

- Urgency performance
- Owner/routing performance
- Action selection
- Critical-issue recall and missed-escalation rate
- False-escalation rate
- Automation coverage versus error rate
- Human-intervention rate
- End-to-end action success
- Latency and model cost

Majority-class and TF-IDF/logistic-regression baselines are good. The model-only, rules-only, and combined ablation is also worth keeping.

However, maintainer-label agreement and policy safety are different evaluations. A deterministic rule might correctly escalate an unusual issue even when its repository label is not

critical. Such a result should not automatically count as a model error.

If we implement `draft_reply`, repository labels cannot grade response quality. That output will require a small human-reviewed rubric, or it should initially use deterministic templates.

The implementation stack is over-specified

Several proposed dependencies are probably unnecessary for the first version:

- Instructor may be redundant if the selected model SDK already provides reliable schema-constrained output.
- MLflow may be excessive for a project with a small number of reproducible evaluation runs. Versioned JSON/CSV results may be enough initially.
- Sentry, structlog, MLflow, UptimeRobot, GitHub Actions, Render, Neon, and several other services all at once create integration work without proving the central idea.
- Using uptime monitoring to prevent a free host from sleeping may be unreliable or contrary to platform expectations.
- Free-tier limits and provider availability are time-sensitive and must be rechecked before choosing infrastructure.

FastAPI, Pydantic, SQLAlchemy, PostgreSQL, server-rendered HTML, and plain Python policy functions remain reasonable. LangGraph is still unnecessary for this fixed workflow.

The claim that the state machine is “about 15 lines” is also too optimistic. The transitions may be small, but reliable execution still needs:

- Transactions

- Idempotency
- Duplicate-event protection
- Failure states
- Retry behavior
- Optimistic concurrency for reviewers
- An immutable audit history

It can remain hand-written without pretending the surrounding reliability work does not exist.

Missing product decisions

Before implementation, we still need to define:

- Who the hypothetical customer and actual user are.
- What volume and operational pain we are designing for.
- Which decisions may execute automatically.
- Which context the system looks up.
- What the owner taxonomy is.
- What “resolution” means in this project.
- What business outcome we will measure.
- Whether the live demonstration uses a form, GitHub webhook, imported issue, or all three.
- Where actions execute safely—ideally a controlled repository or dry-run sandbox, not Kibana itself.

We should not invent claims such as “six maintainers receive 120 issues per week” unless clearly labeled as a fictional scenario or supported by real data.

Scope and timeline

The earlier one-to-two-week estimate is aggressive for a complete, deployed, evaluated system. The project should be divided into milestones:

1. One complete issue through intake, classification, policy, review, and dry-run action.
2. Historical ingestion and label audit.
3. Evaluation and baselines.
4. Deployment and public demo safety.
5. Failure analysis and portfolio packaging.

That sequence reduces the risk of spending the first week building a large evaluation pipeline before proving the actual workflow.

What should remain unchanged

These are sound decisions:

- Use GitHub issue triage as the concrete domain.
- Keep model judgment separate from deterministic policy.
- Use structured outputs.
- Persist workflow state and audit history.
- Prefer ordinary application code over LangGraph.
- Build a minimal server-rendered review interface.
- Use historical replay with real failure analysis.
- Compare the LLM against simple baselines.
- Make missed escalations a primary safety metric.
- Keep the public demo read-only or safely sandboxed.
- Use issue-triage as the honest project name.

The biggest correction is therefore not changing the project. It is ensuring that we build an operational workflow first and use historical classification as its evaluation system—not mistake the evaluation system for the entire product.

Worked for 1m 50s · done 12:37 PM

› Okay, I'm cool with all the changes that you have suggested in the table where we said "recommended corrections." I'm completely fine with those.

The one thing I need you to generally check is whether you have GitHub connection access to GitHub through my GitHub CLI. First, let me know if you don't have access. If you have access

to GitHub, I need you to look into that Kibana Git repo that we have been discussing and see whether that particular dataset is worth using or not. Analyze each and every thing regarding that repository and get all of the pushes. Let me know whether it's worth it or not. If you feel like it's not worth it, I need you to find me the best Git repo that will be useful for this particular task.

- I'm using the agent-reach workflow because this requires live GitHub access and repository research. I'll first verify whether the GitHub CLI is authenticated; if it is, I'll audit Kibana's issue taxonomy, label history, usable sample size, data leakage risks, and suitability for the workflow before comparing alternatives.

- **Explored**

- └ Read SKILL.md (agent-reach skill), dev.md

- Using agent-reach, platform GitHub via the authenticated gh CLI. I'm checking the connection before making any repository queries.

- **Ran** `gh auth status && gh api /user --jq '{login: .login, type: .type}'`
└─ github.com
X Failed to log in to github.com account Ch-Suharsha (default)
... +2 lines (ctrl + t to view transcript)
 - To re-authenticate, run: `gh auth login -h github.com`
 - To forget about this account, run: `gh auth logout -h github.com -u Ch-Suharsha`

- I currently do **not** have authenticated GitHub access.

The GitHub CLI found account Ch-Suharsha, but its token is invalid. Please run:

```
gh auth login -h github.com
```

Once authentication completes, tell me and I'll verify access, then perform the full Kibana repository and issue-dataset audit before recommending whether to use it or replace it.

done 12:44 PM

– Conversation recap

The objective is to apply the approved “recommended corrections”; all proposed changes were accepted. Work is blocked by an invalid GitHub token—run `gh auth login -h github.com`, then clarify the truncated final request.

› i have done the authentication try again

- I'll recheck authentication now. If it succeeds, I'll immediately begin the Kibana audit using live GitHub metadata and issue data.

- **Ran** `gh auth status && gh api /user --jq '{login: .login, type: .type}'`
└─ github.com
X Failed to log in to github.com account Ch-Suharsha (default)
... +2 lines (ctrl + t to view transcript)
 - To re-authenticate, run: `gh auth login -h github.com`
 - To forget about this account, run: `gh auth logout -h github.com -u Ch-Suharsha`

- Authentication still reports invalid. I'm checking whether a stale GH_TOKEN or GITHUB_TOKEN environment variable is overriding the login you just completed; I won't display any token values.

- **Ran** for name in GH_TOKEN GITHUB_TOKEN GH_ENTERPRISE_TOKEN GITHUB_ENTERPRISE_TOKEN; do if [[-n \${P}name]]; then echo "\$name=set"; else echo "\$name=unset"; fi; done; gh config list -h
 - └─ github.com
 - └─ GH_TOKEN=unset
 - └─ GITHUB_TOKEN=unset
 - └─ ... +11 lines (ctrl + t to view transcript)
 - └─ accessible_prompter=disabled
 - └─ spinner=enabled

- **Ran** token_chars=\$(gh auth token -h github.com 2>/dev/null | wc -c | tr -d ' '); echo "stored_token_characters=\$token_chars"; gh api rate_limit --jq '.rate | {limit,remaining,reset}'
 - └─ error connecting to api.github.com
 - └─ check your internet connection or <https://githubstatus.com>

✓ You **approved** codex to always run commands that start with gh auth status

- **Ran** gh auth status && gh api /user --jq '{login: .login, type: .type}'
 - └─ github.com
 - └─ ✓ Logged in to github.com account Ch-Suharsha (keyring)
 - └─ ... +3 lines (ctrl + t to view transcript)
 - └─ - Token scopes: 'gist', 'read:org', 'repo', 'workflow'
 - └─ {"login":"Ch-Suharsha","type":"User"}

- GitHub access is working for account Ch-Suharsha. I'm now auditing Kibana at the level that matters for this project: repository health, live label definitions, issue-label coverage, historical consistency, intake-time leakage, class balance, owner taxonomy, bot noise, and whether critical cases are actually measurable.

✗ You **canceled** the request to run gh repo view elastic/kibana --json nameWithOwner,description,url,homepageUrl,...

- **Ran** gh repo view elastic/kibana --json
 - └─ nameWithOwner,description,url,homepageUrl,isArchived,createdAt,updatedAt,pushedAt, stargazerCount,forkCount,licenseInfo,defaultBranchRef,primaryLanguage,issues,pullRequests && gh api

```
| --paginate 'repos/elastic/kibana/labels?per_page=100' --jq '[] | select((.name |
ascii_lowercase | startswith("impact:")) or (.name | ascii_lowercase | startswith("team:")) or
(.name
| ... +1 lines
| (no output)
```

■ Conversation interrupted - tell the model what to do differently. Something went wrong?
Hit `/feedback` to report the issue.

› hey resume from where you left

- Resuming from the GitHub audit. The previous combined request stalled while paginating Kibana's very large label set, so I'm splitting the audit into bounded queries and will report progress as each section completes.

✓ You **approved** codex to always run commands that start with gh api

- **Ran** gh api repos/elastic/kibana --jq

```
{full_name,description,html_url,archived,disabled,visibility,default_branch,created_at,updated_at,pushed_at,size,stargazers_count,forks_count,open_issues_count,license:.license.spdx
_id,has_issues,has_discussions}'
{"archived":false,"created_at":"2013-01-26T04:00:59Z","default_branch":"main","description":"Your window into all of your
data","disabled":false,"forks_count":8631,"full_name":"elastic/kibana","has_discussions":true,"has_issues":true,"html_url":"https://github.com/elastic/
kibana","license":"NOASSERTION","open_issues_count":14617,"pushed_at":"2026-09-19T19:42:08Z","size":17444777,"stargazers_count":21294,"updated_at":"2026-09-19T19:42:14Z","visibility":"
public"}
```

- **Ran** gh api 'repos/elastic/kibana/labels?per_page=100&page=1' --jq

```
{count:length,relevant:[.[] | select((.name|ascii_lowercase|startswith("impact:")) or
(.name|ascii_lowercase|
| startswith("team:")) or
(.name|ascii_lowercase|test("^ (bug|enhancement|docs|documentation|question)$")))] |
{name,description}}'
{"count":100,"relevant":[]}
```

- **Ran** `gh api --paginate --slurp 'repos/elastic/kibana/labels?per_page=100' --jq 'add | {total:length,relevant:[.] | select((.name|ascii_lowercase|startswith("impact:")) or (.name|ascii_lowercase|startswith("team:")) or (.name|ascii_lowercase|test("^ (bug|enhancement|docs|documentation|question)$"))) | {name,description}}}'`

└ the `--slurp` option is not supported with `--jq` or `--template`

... +17 lines (ctrl + t to view transcript)

--verbose Include full HTTP request and response in the output

- **Ran** `gh api --paginate --slurp 'repos/elastic/kibana/labels?per_page=100' | jq 'add | {total:length,relevant:[.] | select((.name|ascii_lowercase|startswith("impact:")) or (.name|ascii_lowercase|startswith("team:")) or (.name|ascii_lowercase|test("^ (bug|enhancement|docs|documentation|question)$"))) | {name,description}}}'`

└ error connecting to api.github.com

check your internet connection or <https://githubstatus.com>

jq: error (at <stdin>:0): Cannot iterate over null (null)

✓ You **approved** codex to always run commands that start with `jq 'add |`

`{total:length,relevant:[.] | select((.name|ascii_lowercase|startswi...`

- **Ran** `gh api --paginate --slurp 'repos/elastic/kibana/labels?per_page=100' | jq 'add | {total:length,relevant:[.] | select((.name|ascii_lowercase|startswith("impact:")) or (.name|ascii_lowercase|startswith("team:")) or (.name|ascii_lowercase|test("^ (bug|enhancement|docs|documentation|question)$"))) | {name,description}}}'`

└ {

 "total": 1661,

... +505 lines (ctrl + t to view transcript)

]

}

- **Ran** `gh api graphql -f query='query { all: search(query: "repo:elastic/kibana is:issue is:closed created:>=2023-01-01", type: ISSUE, first: 1) { issueCount } labeledImpact: search(query: "repo:elastic/kibana is:issue is:closed created:>=2023-01-01 (label:\\"impact:critical\\" OR label:\\"impact:high\\" OR label:\\"impact:medium\\" OR label:\\"impact:low\\")", type: ISSUE,`

 first: 1) { issueCount } critical: search(query: "repo:elastic/kibana is:issue is:closed

created:>=2023-01-01 label:\\"impact:critical\\", type: ISSUE, first: 1) { issueCount } high:

 ... +8 lines

L

```
{"all":37137,"bug":8833,"critical":434,"criticalFailingTest":69,"docs":146,"enhancement":229,"high":1897,"labeledImpact":0,"low":1607,"medium":2091,"needs":99,"question":41}
```

- The first evidence is mixed: Kibana is highly active and has 1,661 labels, including useful impact labels—but its ownership taxonomy is extremely large and contains many deprecated/

duplicate team labels. Since 2023, 37,137 issues were closed; only about 6,000 carry one of the four scored impact labels. I'm now measuring overlap, year-by-year stability, category

coverage, and how much of that data is actually usable at intake time.

- **Ran** gh api graphql -f query='query { c23: search(query:"repo:elastic/kibana is:issue is:closed created:2023-01-01..2023-12-31

label:\"impact:critical\",type:ISSUE,first:1){issueCount}

| c24: search(query:\"repo:elastic/kibana is:issue is:closed created:2024-01-01..2024-12-31 label:\"impact:critical\",type:ISSUE,first:1){issueCount} c25:

search(query:\"repo:elastic/

| kibana is:issue is:closed created:2025-01-01..2025-12-31

label:\"impact:critical\",type:ISSUE,first:1){issueCount} c26:

search(query:\"repo:elastic/kibana is:issue is:closed

| ... +17 lines

```
{\"c23\":113,\"c24\":114,\"c25\":159,\"c26\":48,\"ch\":2,\"cl\":0,\"cm\":0,\"h23\":439,\"h24\":490,\"h25\":694,\"h26\":274,\"hl\":0,\"hm\":1,\"l23\":518,\"l24\":385,\"l25\":480,\"l26\":224,\"m23\":616,\"m24\":614,\"m25\":524
```

```
,\"m26\":337,\"ml\":3}
```

- **Ran** for impact in critical high medium low; do

| for yr in 2023 2024 2025 2026; do

| if [[\"\$yr\" == \"2026\"]]; then end_date=\"2026-09-19\"; else end_date=\"\$yr-12-31\"; fi

| ... +31 lines

| error connecting to api.github.com

check your internet connection or https://githubstatus.com

| ... +29 lines (ctrl + t to view transcript)

check your internet connection or https://githubstatus.com

jq: error (at <stdin>:0): null (null) and number (0) cannot be divided

- **Running** for impact in critical high medium low; do

| for yr in 2023 2024 2025 2026; do

| if [[\"\$yr\" == \"2026\"]]; then end_date=\"2026-09-19\"; else end_date=\"\$yr-12-31\"; fi

| ... +31 lines

⚠ Heads up, you have less than 25% of your 5h limit left. Run /status for a breakdown.

✓ You **approved** codex to run for impact in critical high medium low; do ... **this time**

- The full issue-level pull is running now, partitioned by year and impact label so GitHub's 1,000-result search cap cannot silently truncate the dataset. This will let me measure unique

issues, multi-label ambiguity, category and owner coverage, bot-generated noise, and taxonomy drift rather than relying on headline counts.

- **Ran** `gh api graphql -F q='repo:elastic/kibana is:issue is:closed created:2026-01-01..2026-09-19 label:"impact:critical" -f`

`query=query($q:String!){search(query:$q,type:ISSUE,first:3)`

`| {issueCount nodes{... on Issue{number title createdAt labels(first:20){nodes{name}}}}}' -jq '.data.search'`

`└ {"issueCount":48,"nodes":[{"createdAt":"2026-08-14T08:51:37Z","labels":{"nodes":[{"name":"bug"}, {"name":"triage_needed"}, {"name":"impact:critical"}, {"name":"Team: SecuritySolution"},`

`{"name":"Team:Entity Analytics"}, {"name":"v9.6.0"}]}, "number":285107,"title":"[Entity Analytics] [Bug] Entities from an existing Kibana space are visible in a newly created space"},`

`... +2 lines (ctrl + t to view transcript)`

`{"createdAt":"2026-06-23T09:22:27Z","labels":{"nodes":[{"name":"bug"}, {"name":"impact:critical"}, {"name":"Team:Fleet - DEPRECATED"},`

`{"name":"QA:Validated"}]}, "number":274504,"title":"[Fleet]: OTel Collector Diagnostics Request Remains Stuck in \"In Progress\" State"}]}`

- **Ran** `q='repo:elastic/kibana is:issue is:closed created:2026-01-01..2026-09-19`

`label:"impact:critical"; gh api graphql --paginate -F q="$q" -f`

`query=query($endCursor:String,$q:String!)`

`| {search(query:$q,type:ISSUE,first:100,after:$endCursor){pageInfo{hasNextPage endCursor}nodes{... on Issue{number title createdAt labels(first:100){nodes{name}}}}}' --jq`

`| '.data.search.nodes[] | {number,title,createdAt,labels:[.labels.nodes[].name]} | jq -s '{count:length, first:[0]}'`

`└ error connecting to api.github.com`

`check your internet connection or https://githubstatus.com`

`... +2 lines (ctrl + t to view transcript)`

`"first": null`

`}`

✓ You **approved** codex to run `q='repo:elastic/kibana is:issue is:closed created:2026-01-01..2026-09-19 labe...` **this time**

- **Ran** `q='repo:elastic/kibana is:issue is:closed created:2026-01-01..2026-09-19 label:"impact:critical"; gh api graphql --paginate -F q="$q" -f query='query($endCursor:String,$q:String!) {search(query:$q,type:ISSUE,first:100,after:$endCursor){pageInfo{hasNextPage endCursor}nodes{... on Issue{number title createdAt labels(first:100){nodes{name}}}}}' --jq | '.data.search.nodes[] | {number,title,createdAt,labels:[.labels.nodes[].name]} | jq -s '{count:length, first:[0]}'`
`{`
 `"count": 48,`
 `... +12 lines (ctrl + t to view transcript)`
`}`
`}`

- **Ran** `gh api -X GET search/issues -f q='repo:elastic/kibana is:issue is:closed created:2026-01-01..2026-09-19 label:"impact:critical" -f per_page=100 --jq '{total_count,items:[.items[] | {number,title,created_at,labels:[.labels[].name]}]}'`
`{`
 `"items": [{"created_at": "2026-08-14T08:51:37Z", "labels": ["bug", "triage_needed", "impact:critical", "Team: SecuritySolution", "Team: Entity Analytics", "v9.6.0"], "number": "285107", "title": "[Entity Analytics] [Bug] Entities from an existing Kibana space are visible in a newly created`
`... +82 lines (ctrl + t to view transcript)`
`users"}, {"created_at": "2026-01-08T22:07:59Z", "labels": ["Team: Presentation", "loe:medium", "impact:critical", "Project: Dashboards API"], "number": "248364", "title": "[dashboard as code] Removing the enhancements registry and all associated code"}], "total_count": 48}`

✓ You **approved** codex to run `gh api --paginate -X GET search/issues -f q='repo:elastic/kibana is:issue is:... this time`

- **Ran** `gh api --paginate -X GET search/issues -f q='repo:elastic/kibana is:issue is:closed created:2025-01-01..2025-12-31 label:"impact:high" -f per_page=100 --jq '.items[] | {number,title,created_at,closed_at,author:.user.login,author_type:.user.type,author_association,comments,labels:[.labels[].name]} | jq -s '{count:length,unique:(unique_by(.number)| length)}'`
`{`
 `"count": 694,`
 `"unique": 694`
`}`

- **Ran** for impact in critical high medium low; do

```

    for yr in 2023 2024 2025 2026; do
      if [[ "$yr" == "2026" ]]; then end_date="2026-09-19"; else end_date="$yr-12-31"; fi
    ... +31 lines
  {
    "total_unique": 6023,
    ... +159 lines (ctrl + t to view transcript)
  }
}

```

✓ You **approved** codex to run for impact in critical high medium low; do ... **this time**

- **Running** for impact in critical high medium low; do

```

    for yr in 2023 2024 2025 2026; do
      if [[ "$yr" == "2026" ]]; then end_date="2026-09-19"; else end_date="$yr-12-31"; fi
    ... +29 lines

```

- A spot check has already exposed an important problem: Kibana's impact:critical is not equivalent to a P1 incident. Recent "critical" items include roadmap/API work, accessibility defects, and CI test failures alongside genuine production-impact bugs. I'm quantifying how widespread that mismatch is before deciding whether Kibana can be repaired into a defensible dataset.

- **Ran** for impact in critical high medium low; do

```

    for yr in 2023 2024 2025 2026; do
      if [[ "$yr" == "2026" ]]; then end_date="2026-09-19"; else end_date="$yr-12-31"; fi
    ... +29 lines

```

gh: API rate limit exceeded for user ID 128779277. If you reach out to GitHub Support for help, please include the request ID EBA1:296D54:1C1753:21DC67:6AAEEF48 and timestamp

2026-09-19 20:23:36 UTC. For more on scraping GitHub and how it may affect your rights, please review our Terms of Service

... +21 lines (ctrl + t to view transcript)

(<https://docs.github.com/en/site-policy/github-terms/github-terms-of-service>) (HTTP 403)

jq: error (at <stdin>:2771): Cannot iterate over null (null)

✓ You **approved** codex to always run commands that start with for impact in critical high; do q="repo:elastic/kibana is:issue is:closed cre...

- **Ran** for impact in critical high; do q="repo:elastic/kibana is:issue is:closed created:2026-01-01..2026-09-19 label:\"impact:\$impact\""; gh api --paginate -X GET search/issues -f q="\$q"

```
| -f per_page=100 --jq '.items[] |
{number,title,created_at,comments,labels:[.labels[]|name]}; done | jq -s
'{count:length,unique:(unique_by(.number))|length}}'
└─ gh: API rate limit exceeded for user ID 128779277. If you reach out to GitHub Support for
help, please include the request ID EBF6:297ACC:1A8F0A:2058BB:6AAEEF56 and
timestamp
2026-09-19 20:23:50 UTC. For more on scraping GitHub and how it may affect your rights,
please review our Terms of Service
... +6 lines (ctrl + t to view transcript)
  "unique": 1
}
```

- Permissions updated to Full Access

⚠ The render MCP server is not logged in. Run `codex mcp login render`.

- **Ran** `gh api rate_limit --jq`

```
{core:.resources.core,search:.resources.search,graphql:.resources.graphql}'
└─
{"core":{"limit":5000,"remaining":4952,"reset":1789851101,"used":48},"graphql":{"limit":500
0,"remaining":5000,"reset":1789853037,"used":0},"search":
{"limit":30,"remaining":30,"reset":1789849497,"used":0}}
```

⚠ MCP startup incomplete (failed: render)

- GitHub's dedicated search endpoint briefly hit its 30-requests-per-minute limit during the exhaustive pull; the account's normal API quota remains healthy. I'm switching to the repository's regular issue endpoint, which avoids that search cap and lets me continue the full audit without guessing from a small sample.

- **Ran** `gh api -X GET repos/elastic/kibana/issues -f state=closed -f labels='impact:critical' -F per_page=100 -F page=1 -f sort=created -f direction=desc --jq '{count:length,newest:.[0].created_at,oldest:[-1].created_at,issues_only:[.[]|select(has("pull_request"))|not]|length}}'`

```
{count:100,issues_only:69,newest:"2026-08-14T08:51:37Z",oldest:"2025-10-30T19:25:15Z"}
```

✓ You **approved** codex to run `set -o pipefail` ... **this time**

- **Running** `set -o pipefail`

```
for impact in critical high medium low; do
  for page_no in {1..100}; do
    ... +42 lines
  └─ impact=critical page=1 oldest=2025-10-30T19:25:15Z
```

```
impact=critical page=2 oldest=2025-02-25T14:39:26Z
... +10 lines (ctrl + t to view transcript)
impact=high page=7 oldest=2025-06-24T09:12:53Z
impact=high page=8 oldest=2025-05-02T14:41:27Z
```

⚠ Heads up, you have less than 10% of your 5h limit left. Run /status for a breakdown.

- The unrestricted repository endpoint is now enumerating the scored issues. I'm also separating issues from pull requests—an important correction, because roughly one-third of the first

impact:critical API page consisted of pull requests carrying the same label. Those must never enter the triage dataset.

- Model changed to gpt-5.6-sol low

- **Ran** gh api rate_limit --jq

```
{'core':.resources.core,search:.resources.search,graphql:.resources.graphql}'
```

```
{'core':{'limit':5000,'remaining':4913,'reset':1789851101,'used':87},'graphql':{'limit':5000,'remaining':5000,'reset':1789853160,'used':0},'search':{'limit':30,'remaining':30,'reset':1789849620,'used':0}}
```

- **Ran** set -o pipefail

```
for impact in critical high medium low; do
  for page_no in {1..100}; do
    ... +42 lines
  impact=critical page=1 oldest=2025-10-30T19:25:15Z
  impact=critical page=2 oldest=2025-02-25T14:39:26Z
  ... +244 lines (ctrl + t to view transcript)
}
}
```

✓ You **approved** codex to run for yr in 2023 2024 2025 2026; do ... **this time**

- **Ran** for yr in 2023 2024 2025 2026; do

```
if [[ "$yr" == "2026" ]]; then end_date="2026-09-19"; else end_date="$yr-12-31"; fi
gql_query='query {'
... +44 lines
gh: Resource limits for this query exceeded.
Resource limits for this query exceeded.
... +3094 lines (ctrl + t to view transcript)
Resource limits for this query exceeded.
jq: error (at <stdin>:0): Cannot iterate over null (null)
```

- The oversized GraphQL query exceeded GitHub's query-complexity limit, so I'm abandoning that route. I'm switching to smaller, auditable page pulls saved only in /tmp; this is slower but avoids silent truncation and preserves exact issue-level evidence. No project files are being changed.

✓ You **approved** codex to run audit_file=/tmp/kibana-impact-audit.jsonl ... **this time**

- **Ran** audit_file=/tmp/kibana-impact-audit.jsonl

```
truncate -s 0 "$audit_file"
impact=critical
... +12 lines
impact=critical page=1 oldest=2025-10-30T19:25:15Z
impact=critical page=2 oldest=2025-02-25T14:39:26Z
... +8 lines (ctrl + t to view transcript)
"oldest": "2023-01-09T15:47:53Z"
}
```

/status

>_ **OpenAI Codex** (v0.155.1)

Visit <https://chatgpt.com/codex/settings/usage> for up-to-date information on rate limits and credits

Model: gpt-5.6-sol (reasoning low, summaries auto)

Model provider: openai

Directory: ~/Documents/projects/issue-triage

Permissions: Full Access

Agents.md: <none>

Account: suharsha.cheedalla@sjsu.edu (Edu Plus)

Thread name: Analyze project chat

Collaboration mode: Default

Session: 01a0bb14-a0ef-7e61-992a-0a93d136155a

Context window: 58% left (115K used / 258K)

5h limit: 7% left (resets 17:13)

Weekly limit: 75% left (resets 12:37 on 23 Sep)

✓ You **approved** codex to run audit_file=/tmp/kibana-impact-audit.jsonl ... **this time**

- **Ran** audit_file=/tmp/kibana-impact-audit.jsonl
| impact=high
| for page_no in {1..40}; do
| ... +11 lines
|
| impact=high page=1 oldest=2026-06-12T12:27:24Z
| impact=high page=2 oldest=2026-03-30T10:57:17Z
| ... +23 lines (ctrl + t to view transcript)
| "high": 1897
| }

- **Running** audit_file=/tmp/kibana-impact-audit.jsonl
| impact=medium
| for page_no in {1..40}; do
| ... +11 lines

⚠ Heads up, you have less than 5% of your 5h limit left. Run /status for a breakdown.

✗ You **canceled** the request to run audit_file=/tmp/kibana-impact-audit.jsonl ...

- **Ran** audit_file=/tmp/kibana-impact-audit.jsonl
| impact=medium
| for page_no in {1..40}; do
| ... +11 lines
|
| (no output)

■ Conversation interrupted - tell the model what to do differently. Something went wrong?
Hit `/feedback` to report the issue.

› I've seen that the limits are getting to an end, so the best way to continue is to let some other model handle it from here. I don't think we can use any other model from Codex, so whatever you have done till now (not everything in a Markdown file), I'll just hand the task to a different coding agent.

- That's fine. Here is the important handoff state for the next coding agent.

GitHub access is working through gh for account Ch-Suharsha. No project files were modified; only temporary audit data was created under /tmp.

Current project decisions:

- Build an AI-assisted issue-triage workflow.
- Separate model judgment from deterministic policy code.
- Use structured outputs, persistent workflow state, human approval, audit logs, and historical replay.
- Treat GitHub data as the evaluation corpus, not the entire product.
- Keep ask_info, draft_reply, route, and escalate as business actions.
- Model state, approval state, and workflow state must remain separate.
- The public demo should be safe/read-only or dry-run.

Kibana findings so far:

- Repository: elastic/kibana
- Active, public, very large, and heavily maintained.
- Approximately 21k stars, 8.6k forks, and 14.6k open issues at audit time.
- The repository has 1,661 labels.
- Its team taxonomy is very large, inconsistent, and contains many deprecated/duplicate labels.
- Closed issues created since 2023: approximately 37,137.
- Impact-label counts since 2023:
 - impact:critical: 434
 - impact:high: 1,897
 - impact:medium: 2,091
 - impact:low: 1,607
 - impact:needs-assessment: 99
- Some issues carry multiple impact labels, so those counts are not perfectly disjoint.
- The GitHub /issues endpoint includes pull requests; the first critical-impact page contained 31 PRs out of 100 results. PRs must be filtered out.
- impact:critical is not equivalent to “security incident” or necessarily P1. Recent critical examples included:
 - ordinary product bugs,
 - accessibility defects,
 - API/roadmap work,
 - CI “Failing test” issues,
 - security-related bugs.
- A search found 69 critical issues with titles beginning “Failing test.”
- The word “Security” is unsafe as a security rule because Kibana has multiple Security products and teams.

Preliminary conclusion:

Do not use raw Kibana labels as unquestioned ground truth. Kibana may still be usable after strict filtering and a label-quality audit, but its raw dataset is noisy and its impact labels

do not cleanly represent urgency, security, or routing.

The audit was not fully completed. Critical and high-impact issue data was collected; the medium-impact pull was interrupted, so no final Kibana-versus-alternative recommendation was

reached. The next agent should:

1. Finish the medium/low issue audit.
2. Measure category and owner coverage after filtering.
3. Check label consistency over time.
4. Use `created_at` rather than `closed_at` for temporal evaluation.
5. Build separate operational and critical-safety test sets.
6. Compare Kibana against alternative repositories before committing.

No code has been built yet.

done 1:34 PM

- Model changed to gpt-5.6-sol medium

› I need you to build me a new file, naming it `handout.md`, explaining each and everything regarding what we had done till now and the task, including where it needs to resume. Tell it about the handouts. I'm going to keep the PDF regarding the earlier chat and every piece of information. Write in detail, in depth, about each and every small, tiny thing that you have discovered and you planned to verify, which you couldn't, so that you could verify it. Build this particular `handout.md` file and let me know once it's done.

- I'll create a detailed handoff document that separates confirmed decisions, provisional recommendations, verified GitHub findings, incomplete checks, known data-quality risks, and the

exact restart point for the next agent. I'll first inspect the workspace and the partially collected audit data so the handoff does not overstate what was completed.

- **Ran** `rg --files -g '!tmp/**' -g '!output/**' | sed -n '1,160p'; if [[ -f /tmp/kibana-impact-audit.jsonl ]]; then wc -l -c /tmp/kibana-impact-audit.jsonl; jq -s '{rows:length,unique:`

`|`
`(unique_by(.number)|length),impact_counts:([.[]|.labels[]|select(startswith("impact:"))]|gro`
`up_by(.)|map({key:.[0],value:length})|from_entries),oldest:(map(.created_at)|min),newest:`

```
| (map(.created_at)|max)}' /tmp/kibana-impact-audit.jsonl; else echo 'temporary audit file  
not present'; fi  
L{  
  "rows": 2331,  
  ... +8 lines (ctrl + t to view transcript)
```